

National Undergraduate Programme in Mathematical Sciences
Introduction to Programming

Mid-semester Examination, Semester I, 2024-2025

Date : September 30, 2025

Time : 0930-1230

Maths 101
11/22/2025 10/11

- This paper has two parts.
- There are 8 questions in Part A, each worth 5 marks. For this part, provide the *solution* and a short explanation justifying your answer.
- Part B is worth 60 marks. For B2 and B3, provide crisp and short programs. Syntax should more or less correspond to Haskell, but minor inaccuracies like wrong indentation will be ignored. You can assume standard library functions, but if you use something not taught in class, you must explain briefly what it does.

Part A

1. Suppose f , $l1$, $l2$ and $l3$ are defined as follows:

```
f []      ys = ys
f (x:xs) ys = x: f xs ys
```

```
l1 = foldr f [] [[0,1], [2..5], [6..13]]
l2 = foldr1 f    [[0,1], [2..5], [6..13]]
l3 = foldl  f [] [[0,1], [2..5], [6..13]]
```

What are the values of $l1$, $l2$ and $l3$?

2. How many times is the second line of the definition of f invoked in the computation of $l1$?
3. How many times is the second line of the definition of f invoked in the computation of $l2$?
4. How many times is the second line of the definition of f invoked in the computation of $l3$?
5. Give a polymorphic type of the function g defined as follows:

```
g f = foldr (\x l → f x : l) []
```

$g :: (a \rightarrow c) \rightarrow [c]$

6. Recall the definition of **map**, and consider the list $l :: [\text{Int}]$ given below:

```
map f []           = []
map f (x:xs)       = f x: map f xs

l                  = map (*2) (map (+3) xs)
```

How many times (in terms of n , the length of xs) is the second line of the definition of **map** invoked in the computation of l ?

7. Give an alternative definition of l which reduces the above number.
8. Give five values that belong to the type **Form** defined as follows:

```
data Form = Var Integer | Bot | Imp Form Form
```

Part B

1. Given the following definition of $\text{fib} :: \text{Integer} \rightarrow \text{Integer}$, give the complete trace of the computation of $\text{fib } 4$. (Remember that any definition is expanded only if there is a need. To compute $e1 + e2$, one needs to reduce first $e1$ and then $e2$ to a number.)

```
fib n = if n ≤ 1 then n else fib (n-2) + fib (n-1)
```

(15 marks)

2. Define a function varValue whose type and values on sample inputs are given below.

```
varValue :: Integer → [Bool] → Maybe Bool
```

$\text{varValue } 0$	$[\text{False}, \text{True}]$	$= \text{Just False}$
$\text{varValue } 1$	$[\text{False}, \text{True}, \text{True}]$	$= \text{Just True}$
$\text{varValue } 5$	$[\text{False}, \text{True}]$	$= \text{Just False}$
$\text{varValue } 19$	$(\text{replicate } 20 \text{ True})$	$= \text{Just True}$
$\text{varValue } 100$	$(\text{replicate } 50 \text{ False} ++ \text{repeat True})$	$= \text{Just True}$
$\text{varValue } (-1)$	(repeat False)	$= \text{Nothing}$
$\text{varValue } (-20)$	(repeat True)	$= \text{Nothing}$
$\text{varValue } 100$	$(\text{replicate } 50 \text{ True})$	$= \text{Just False}$

(20 marks)

3. The Form data type above represents *propositional logic formulas*, with Bot standing for the always false formula $\perp$, and Imp standing for implication (denoted $\rightarrow$), whose meaning is given as follows:

$$b_1 \rightarrow b_2 = \begin{cases} \text{false} & \text{if } b_1 \text{ is true and } b_2 \text{ is false} \\ \text{true} & \text{otherwise} \end{cases}$$

Define a function eval whose type and values on sample inputs are given below. You must use the varValue function defined above for the base case.¹

eval :: [Bool] → Form → Maybe Bool

let val1 = [False, True]

let val2 = replicate 20 True

let val3 = concat (repeat [False, True])

let val4 = repeat True

eval val1	(Var 0)	= Just False
eval val2	(Var 100)	= Just False
eval val2	(Var (-2))	= Nothing
eval val1	Bot	= Just False
eval val4	Bot	= Just False
eval val3	(Imp (Var 100) Bot)	= Just True
eval val3	(Imp (Var 99) Bot)	= Just False
eval val4	(Imp (Var 1000) Bot)	= Just False
eval val4	(Imp (Imp (Var 1000) Bot) Bot)	= Just True
eval val3	(Imp (Var 99) (Imp (Var 100) Bot))	= Just True
eval val4	(Imp (Var (-1)) (Var 100))	= Nothing

(25 marks)

¹Even if you do not get a correct solution for varValue, if you use it correctly in this function and provide a correct definition of eval, I shall award you marks.